# ConceptFormer-2: Token-Efficient Grounding of Frozen Instruction-Tuned Language Models in Knowledge Graphs

**Joel Barmettler**
University of Zurich
Zurich, Switzerland
joel.barmettler@uzh.ch

**Abraham Bernstein**
Department of Informatics
University of Zurich
Zurich, Switzerland
bernstein@ifi.uzh.ch

**Luca Rossetto**
School of Computing
Dublin City University
Dublin, Ireland
luca.rossetto@dcu.ie

## Abstract

Retrieval grounds language models in external knowledge, but it pays for this grounding on every query with hundreds of context tokens. ConceptFormer-2 instead compresses an entity's Wikidata neighborhood into $k$ continuous concept tokens that a frozen, instruction-tuned language model consumes in place of retrieved text. A small resampler builds these tokens from the frozen model's own label embeddings, so an unseen entity requires one forward pass and no new parameters. Training needs no labels: the frozen model reading $k$ concept tokens is distilled toward the same frozen model reading the facts as text. On PopQA, eight concept tokens lift a frozen Qwen3-0.6B from 0.10 to 0.48 exact match on unseen entities, an accuracy that text-based baselines only reach with 64 to 128 fact tokens. Accuracy grows with token budget and with training data. Larger frozen backbones gain less from injection, and the shortfall is not monotone in model size; both patterns replicate across the Qwen3 and Gemma-3 families. In both, run-to-run variance at the largest corpus overtakes the effects under study. Counterfactual probes confirm that answers follow the injected graph, and the encoder transfers zero-shot to MetaQA, raising accuracy from 0.06 to 0.34 on a knowledge graph it never saw in training. All corpora, checkpoints, and per-item evaluation outputs are released.

## 1 Introduction

Instruction-tuned language models answer questions about popular entities from their weights, but they fail on the long tail, a deficit that persists even in frontier models [40, 41]. Retrieval-augmented generation [42] restores factuality at a recurring cost. A single entity's verbalized Wikidata neighborhood occupies a median of roughly 100 context tokens in our corpora, and on every query this text competes with the user's actual task for the context window.

ConceptFormer [1] proposed a third option: compress the knowledge-graph neighborhood itself into a few continuous tokens that a frozen language model consumes natively. The original system was validated on GPT-2 with rank-based metrics for next-token factual recall, and it left the decisive questions open. Rank-based scores cannot tell whether a model would actually generate a fact, because a gold answer among the top logits can still lose the greedy decode to a competing wrong

Preprint.

one. Whether the model reads the injected structure, or merely absorbs it as an opaque signal, was never tested. And the expectation that injection, like soft prompting [51], would benefit from larger frozen models remained a conjecture. ConceptFormer-2 measures all three. Does latent injection survive generative, exact-match question answering with a modern conversational model? Does the model read the injected graph? And what governs the value of this channel: the token budget, the amount of knowledge the encoder is trained on, or the size of the frozen model?

ConceptFormer-2 addresses these questions with a redesigned system. Three design changes matter most. The featurizer represents each graph edge with the frozen model's own label embeddings instead of v1's per-entity lookup table, which makes the encoder inductive: an unseen entity needs only its labels. The training objective is label-free. The frozen model reading $k$ concept tokens is distilled toward the same frozen model reading the facts as text, with a full-vocabulary KL loss and no annotated QA anywhere in the pipeline. Finally, the encoder's output projection is zero-initialized, so training starts from a student that behaves identically to the frozen model.

**Contributions.** **(1)** A latent knowledge-injection system for frozen instruction-tuned LLMs, evaluated with generative exact match on PopQA against strong text baselines and an untrained-injection control (§5.1). **(2)** Matched scaling measurements along three axes: token budget, training-corpus size, and frozen-model size, showing how the three trade against each other (§5.2). **(3)** Causal probes showing that answers follow the injected graph, and zero-shot transfer of the trained encoder to an unseen knowledge graph (§5.3, §5.4). **(4)** An open release of the corpora, checkpoints, evaluation harness, and per-item outputs, enabling paired re-analysis of every reported comparison.

## 2 Related work

**Lineage.** The problem framing descends from work that probes language models as knowledge bases [43, 44] and its long-tail refinement [41, 40, 2], with RAG [42] as the deployed remedy. Knowledge-enhanced pretrained models such as KnowBERT [45], K-BERT [46], and K-Adapter [47] inject graph signal by modifying the model itself, which our frozen-model constraint rules out. ConceptFormer v1 [1] injected pretrained KG node embeddings (TransE-family vectors at PyTorch-BigGraph scale [48, 49]) through a per-entity lookup table. The present system builds its features from the frozen model's own label embeddings instead, which is what makes the encoder inductive. Prepending learned continuous tokens to a frozen model descends from prefix-tuning and soft prompts [50–52], textual inversion [53], and most directly from Frozen [54] and Flamingo [56]. Our substrate is Wikidata [57], and the verbalization baselines follow the graph-to-text encoding line [58].

**Soft-token context compression.** Gist tokens [24] and AutoCompressor [25] adapt the language model itself. ICAE [23] and 500xCompressor [26] train LoRA encoders against frozen decoders with reconstruction objectives, and CompLLM [28] distills per-segment embeddings through activation matching. ARC-Encoder [29] is the strongest recent text compressor for frozen decoders. IC-Former [3] is architecturally the closest compressor: learnable digest tokens cross-attend over the LLM's word embeddings in a small standalone encoder. A parallel personalization line builds reusable per-user embeddings with resamplers for frozen LLMs [4, 5]; these are per-entity soft tokens in structure, without graph input and without label-free training. PISCO [27] follows the same principle as our objective, distillation from a model that reads the uncompressed text, but implements it with sequence-level distillation on teacher-generated answers and LoRA-tunes both compressor and decoder. It reports that a frozen decoder clearly underperforms at document granularity; at entity granularity, our frozen decoder works well. The closest system is xRAG [22], which injects into the input embeddings of a frozen decoder and includes a KL term against the same model reading the text. Its training additionally relies on paraphrase pretraining and about one million labeled instruction examples. Its representation is a single pre-existing retriever embedding passed through a projector, which binds it to that embedding model at inference. Its evaluation covers aggregate QA accuracy and robustness to noisy retrieval, and it tests neither unseen-entity generalization nor what the compressed representation actually conveys. Cartridges [30] distills each corpus into a trainable KV cache by gradient descent on self-generated conversations. This amortizes over the queries to one corpus, but there is no encoder, so a new corpus requires a fresh optimization run. The same teacher-reads-text principle also appears outside the input space: prompt distillation [6] tunes LoRA adapters toward the same model reading new documents, context distillation refines this in weight space [7], and KV-Distill [8] applies the KL objective to compress KV caches. A systematic study of gist-token compression [9] catalogs what such representations drop, including the counterfactual failures our probes target. None of these systems compress structured input, and none tests the compressed representation causally. Post-hoc overflow probing [31] is the nearest attempt.

**Knowledge-graph injection into LLMs.** GNP [34] and G-Retriever [36] retrieve a question-specific subgraph and run a graph encoder over it at inference time, so every question pays a retrieval and encoding pass. Both train with next-token cross-entropy on labeled tasks, GNP with an auxiliary link-prediction loss, and G-Retriever also feeds the retrieved subgraph as plain text. LGPT [10] refines this family with learnable graph pooling tokens, still computed per question. GraphToken [35] encodes small synthetic graphs for graph-reasoning questions; its encoder never reads the question, but its graphs contain no entities, and it names factual grounding as future work. LLaGA [37] projects frozen text-encoder features of a node's sampled neighborhood into token space and is structurally the closest to per-entity encoding, but it rebuilds the sequence for every instance and targets node classification and link prediction rather than factual recall. All four keep the LLM frozen and train the encoder or projector on task labels. None of them computes entity tokens once and reuses them across arbitrary prompts. KBLaM [32] precomputes one key–value pair per triple with a sentence encoder and learned adapters, injecting them into every attention layer through a modified attention with retrained query heads. New triples need only an encoder pass, but training requires supervised QA pairs, and on the one real knowledge base it is tested on, built from Enron emails, its accuracy degrades. SR-KI [11] scales the same recipe to tens of thousands of triples with a supervised attention loss; its per-triple latents are reusable, but training requires labeled QA and the injected units are flat triples rather than compressed neighborhoods. ReaLM [12] quantizes pretrained KG embeddings into discrete tokens in the LLM vocabulary for knowledge-graph completion, which trains the vocabulary and targets a different task. Knowledge Prompts [33] learns one soft prompt per entity for 1.1M Wikidata entities as free parameters in a lookup table, so covering a new entity means training a fresh prompt; no encoder exists to produce one. Recent analyses find that graph-token LLMs "do not fully understand graph tokens" [38] and that graph tokens can decouple from graph semantics [39]. Our counterfactual evaluation (§5.3) measures this decoupling directly.

## 3 Method

**Setup and featurizer.** Let $M$ be a frozen chat LLM with input-embedding dimension $d$. For an entity $e$ with 1-hop neighborhood $G_e = \{(p_i, n_i)\}_{i=1}^{N}$ (property and neighbor labels, PageRank-ordered), each edge becomes $x_i = [\,\phi(p_i); \phi(n_i)\,] \in \mathbb{R}^{2d}$, where $\phi$ mean-pools $M$'s input embeddings over a label's tokens. The features live in $M$'s own embedding space, and no per-entity parameters exist anywhere in the system.

**Encoder.** The encoder is a latent-query resampler [55, 56]. In each of $L$ layers, $k$ learned queries cross-attend over the edge set (order-invariant, padded edges masked), self-attend, and pass through a feed-forward block ($d_{\text{model}}$=1024, $L$=4, roughly 70M parameters; Table 4 ablates this capacity). An output projection maps the queries to $\mathbb{R}^{k\times d}$. The projection is zero-initialized, so at step 0 the student is bit-identical to the frozen model. Because the encoder input is only the entity's neighborhood, the concept tokens are independent of the question: they can be computed once per entity and cached, and an unseen entity needs a single encoder forward pass.

**Objective.** The teacher is $M$ reading [system][verbalized $G_e$][question], and its greedy answer path $y_{1..m}$ is stored offline. The student is $M$ reading the same question, with the concept block

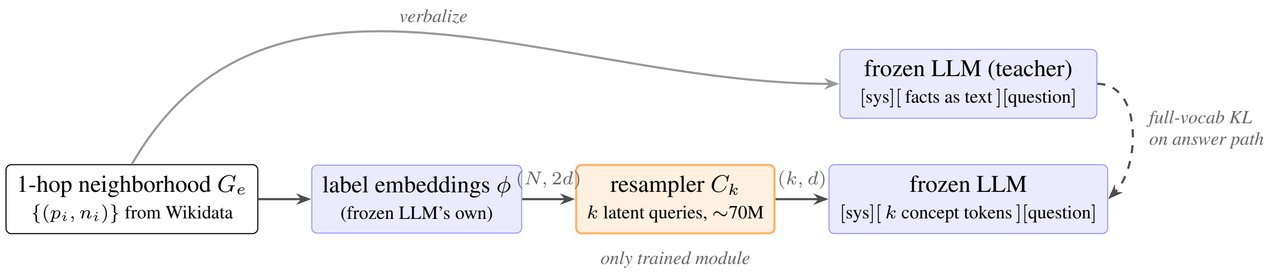


Figure 1: ConceptFormer-2. Edge features are the frozen model's mean-pooled label embeddings, a Perceiver-style resampler compresses the edge set into $k$ concept tokens, and the tokens are spliced into the frozen model's input. The training signal is the KL between the same frozen model reading the facts as text (teacher) and reading the $k$ tokens (student).

$C_k(G_e)$ spliced in immediately before the entity mention at ordinary consecutive token positions. Table 4 ablates this placement; we anchor the block to the mention because a prompt with several entities can then carry one block per mention. The loss is the token-level full-vocabulary KL [60] along the stored path,

$$\mathcal{L} = \frac{1}{m} \sum_{t=1}^{m} \mathrm{KL}\big(p_M(\cdot \mid \text{facts}, q, y_{<t}) \,\|\, p_M(\cdot \mid C_k(G_e), q, y_{<t})\big)\,. \tag{1}$$

Only the roughly 20 path positions pass through the LM head, which keeps training light. One converged run fits a single 24 GB consumer GPU. The corpus questions (generated by a separate LLM, described below) only give the teacher something to answer. No QA labels enter the loss.

**Data.** From the Wikidata PageRank ranking we sample entity pools, excluding all PopQA subjects, and snapshot each entity's complete 1-hop neighborhood. A separate open-weights Gemma model generates questions from these neighborhoods. Each question is then tiered: it enters the corpus if the graph demonstrably helps the backbone, meaning the base model fails the question and the model with the facts in context answers it (a 30% sample of base-known questions is retained). Finally, we store the teacher's greedy answer paths. The corpora hold 10k entities (92,177 rows under the Qwen3-0.6B [61] teacher) and 100k entities (925,178 rows). Tiering and teacher paths depend on the backbone, so each backbone trains on its own corpus variant of near-identical size, and cross-backbone comparisons evaluate each model with its own curriculum.

**Training.** AdamW with constant learning rate $10^{-4}$, weight decay 0.01, and effective batch size 32 through gradient accumulation. We train 72k optimizer steps on the 10k corpus and 100k steps (about 4 epochs) on the 100k corpus, and select the best checkpoint on a held-out sample. Reported configurations use three random seeds unless a table states otherwise.

## 4 Evaluation protocol

Every configuration is scored between two conditions of the same frozen model: **base**, the model without any knowledge, and **RAG**, the model reading the verbalized facts with the questioned edge guaranteed present. Base gives the floor and RAG gives the ceiling (0.96 on PopQA). The metric is greedy exact match with word-boundary alias matching. We additionally report the official PopQA metric, which is more lenient and tracks ours within about 0.5 pt.

We report accuracy on three sets. **Held-out** questions are unseen questions about trained entities. **Held-in** questions repeat training questions and gauge overfitting. **PopQA** [59] asks about entities the encoder never saw and measures inductive generalization ($n$=14,266 questions with snapshot

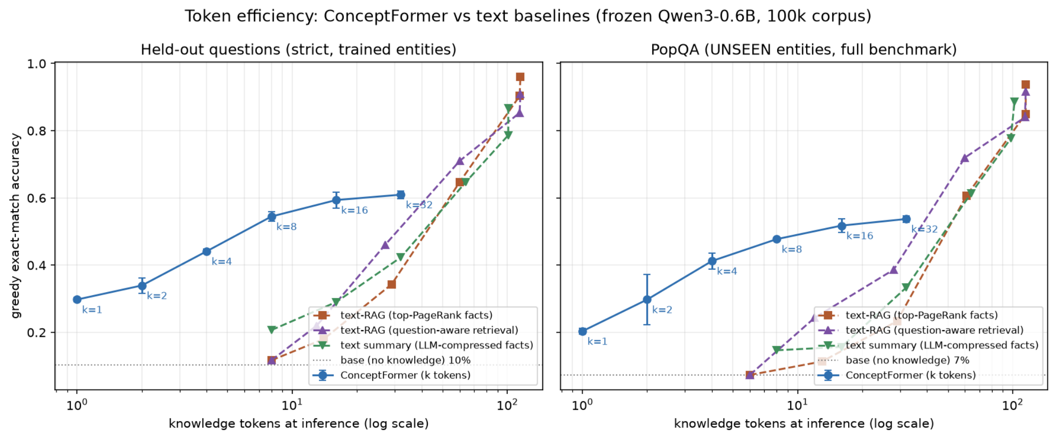


Figure 2: Accuracy against knowledge tokens spent at inference (log scale), frozen Qwen3-0.6B, 100k-entity corpus, 3 seeds. Concept tokens are compared with three text baselines at the same token budget: top-PageRank truncation, question-aware retrieval, and LLM-written summaries. Left: strict held-out. Right: PopQA.

Table 1: The $k$-family on the 100k-entity corpus (frozen Qwen3-0.6B, 3 seeds, mean±std). Strict held-out $n$=2000, PopQA $n$=14,266. The official PopQA metric is reported for comparability with the literature. Base scores 0.103 (official 0.107) and RAG scores 0.960. Each adjacent $k$ step is significant under an item-paired McNemar test ($p < 10^{-5}$).

| $k$ | 1 | 2 | 4 | 8 | 16 | 32 |
|---|---|---|---|---|---|---|
| held-out | .298±.006 | .340±.023 | .441±.007 | .545±.014 | .594±.024 | .610±.011 |
| PopQA | .203±.008 | .298±.075 | .412±.023 | .477±.002 | .518±.020 | .537±.010 |
| PopQA (official) | .207±.009 | .303±.077 | .419±.024 | .483±.002 | .523±.021 | .543±.010 |

coverage). Held-out splits are grouped by fact, so paraphrases of the same (entity, fact) pair never straddle the train/test boundary, and the evaluated subset is further restricted to facts that were never supervised (tables mark this set as strict). All systems are scored on identical item sets, which permits paired statistics: we report Wilson intervals and exact McNemar tests. Checkpoints are selected on one evaluation sample and scored on larger, disjoint sets. Per-item outputs for every evaluation are released.

## 5 Results

### 5.1 Token efficiency against strong text baselines

Figure 2 compares concept tokens with three text baselines at matched knowledge-token budgets: query-independent top-PageRank truncation, question-aware retrieval that ranks facts by embedding similarity to the question, and LLM-written budgeted summaries. The question-aware baseline sees the question and can select the one relevant fact, an advantage the query-independent concept tokens do not have. On unseen entities at a budget of eight tokens, concept tokens reach 0.48 accuracy and the best text baseline reaches 0.15. The text baselines reach the same accuracy only with 64 to 128 fact tokens, so matched accuracy costs text five to six times more tokens. An untrained control separates the encoder from the injection mechanism: filling the same $k$ slots with mean-pooled embeddings of the top-$k$ edges lifts PopQA accuracy only from 0.103 to 0.130. The untrained slots recover 7% of the trained gain, so the effect comes from the learned compression.

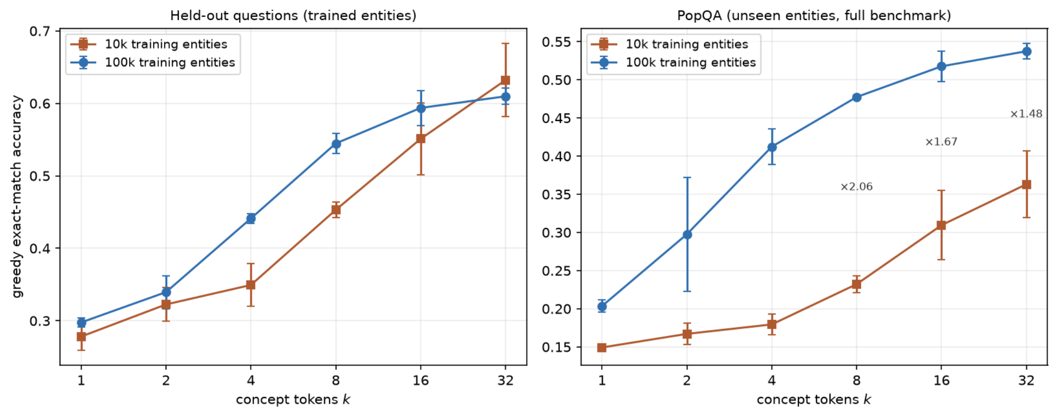


Figure 3: Token-budget curves at two corpus scales (3 seeds, identical protocol). Left: strict held-out. Right: PopQA, with the 10k→100k accuracy ratio annotated per $k$.

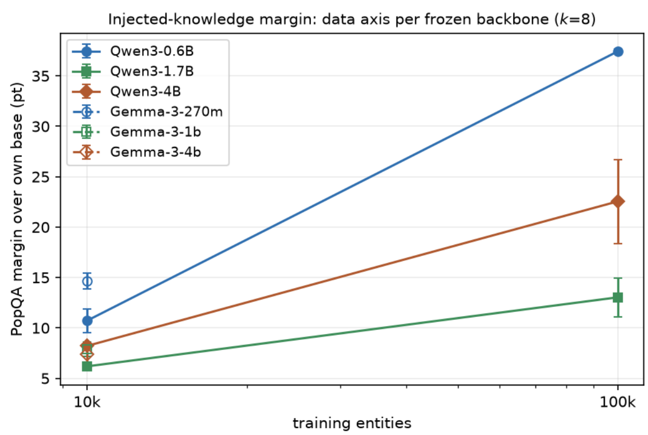


Figure 4: PopQA margin over each frozen backbone's own base accuracy at $k$=8, for both training-corpus sizes. Cell values and seed counts are given in Table 2.

### 5.2 The three-way trade: tokens, data, and model size

**Token axis.** Accuracy is monotone in $k$ through $k$=32 on both corpora (Table 1, Figure 3). Returns per token diminish: $k$=8 reaches about 89% of the $k$=32 PopQA accuracy with a quarter of the tokens. The best budget therefore depends on how much context a deployment is willing to spend, and the curve shows no plateau up to $k$=32.

**Data axis.** Scaling training entities $10\times$ at $k$=8 lifts PopQA accuracy from $0.210$ to $0.477$ on identical evaluation sets (Table 2), and the gain had not saturated when training ended. Held-out accuracy rises as well ($0.416 \to 0.545$) while held-in accuracy falls (about $0.82 \to 0.64$). With more entities, the encoder memorizes individual training entities less and generalizes the edge-to-concept mapping more.

**Token–data substitution.** The two axes interact. The 10k→100k PopQA ratio falls from 2.05 at $k$=8 to 1.48 at $k$=32 (annotations in Figure 3). A larger token budget extracts more value from a small corpus and partially substitutes for training data.

**Model axis.** We repeat the data transect across the Qwen3 family, each backbone with its own teacher-relative corpus and brackets (Figure 4, Table 2). A larger frozen model knows more of PopQA from its weights, so the grid asks whether it gains less from injection and whether more

Table 2: The data $\times$ model grid at $k$=8: full-PopQA accuracy and margin over each frozen backbone's own base (mean±std over seeds). Corpora are teacher-relative (§3), so each backbone trains on its own curriculum of near-identical size.

| backbone | base | 10k entities | 100k entities | data gain | seeds |
|---|---|---|---|---|---|
| Qwen3-0.6B | .103 | .210±.012 (+10.7) | .477±.002 (+37.4) | 3.5× (margin) | 6 / 3 |
| Qwen3-1.7B | .141 | .203±.003 (+6.2) | .271±.019 (+13.0) | 2.1× (margin) | 3 / 3 |
| Qwen3-4B | .154 | .237±.002 (+8.2) | .380±.042 (+22.5) | 2.7× (margin) | 3 / 3 |
| Gemma-3-270m | .039 | .185±.008 (+14.7) | .268±.044 (+22.9) | 1.6× (margin) | 3 / 3 |
| Gemma-3-1b | .112 | .191±.007 (+7.9) | .217±.023 (+10.4) | 1.3× (margin) | 3 / 3 |
| Gemma-3-4b | .167 | .241±.008 (+7.4) | .281±.006 (+11.5) | 1.6× (margin) | 3 / 3 |

training data closes the gap. It gains less, and data does not close the gap. The 1.7B margin is roughly half the 0.6B margin at both corpus scales. The same 10× data step multiplies the 0.6B margin 3.5× and the 1.7B margin 2.1×. Headroom does not explain this: base accuracy moves only from 0.10 to 0.15 across the family and the RAG ceiling stays near 0.97, yet at 100k entities the 0.6B closes 44% of its base-to-RAG gap, the 4B closes 28%, and the 1.7B closes 16%.

Two observations sharpen the picture. The 1.7B held-out accuracy barely moves across the data step ($0.397 \rightarrow 0.418$) where the 0.6B gains 13 points. The encoder trains equally well on the bigger backbone, and what shrinks is specifically the payoff on unseen entities. And the axis is not monotone: the 4B margin exceeds the 1.7B margin at both corpus scales (+8.2 vs. +6.2 at 10k, +22.5 vs. +13.0 at 100k), and its data gain of 2.7× sits between the 0.6B's 3.5× and the 1.7B's 2.1×. The 1.7B is a valley of the model axis rather than a point on a decay. The teacher-relative curricula remain a candidate confound throughout, since a stronger base tiers a different question mix into its training corpus.

A second model family tests how much of this is Qwen-specific. Across Gemma-3 backbones (Table 2), the margin again roughly halves from the smallest model to the mid-size one at both corpus scales, and the smallest backbone in either family posts the largest margin. The mid-size dip also replicates in the data gain: Gemma's 1b multiplies its margin 1.3× under 10× data where its neighbors reach 1.6×, mirroring Qwen's 2.1× valley between 3.5× and 2.7×. The 4B recovery is clear in Qwen at both scales; in Gemma the 4b edges above the 1b only at 100k, and within noise. Gemma's data gains are uniformly smaller than Qwen's, so the value of additional training data itself depends on the family.

**Stability across scale.** Instability emerges at the larger corpus in both families, but on different backbones. Every 10k cell in either family is reproducible to within ±0.8 points. At 100k, the seed spread reaches ±1.9 and ±4.2 on Qwen's two larger backbones and ±4.4 and ±2.3 on Gemma's two smaller ones, while Qwen3-0.6B and Gemma-3-4b stay within ±0.6. Run-to-run variance is therefore not a monotone function of model size: it is a property of the backbone–corpus pair. Where it strikes, it swamps effects that stable cells resolve cleanly; doubling the 1.7B's token budget at 100k moves the mean margin from +13.0 to +15.1, less than the seed spread of either cell. Stabilizing the recipe per backbone is a prerequisite to measuring such effects at scale.

### 5.3 Causal probes of graph reading

A compression system can score well on QA while treating the graph as an opaque signal, a deficiency diagnosed across graph-token systems [38, 39]. Existing checks are correlational: diagnostic probes on resampler tokens in vision-language systems [15] and controlled edits of textual reasoning structures [16]. We instead intervene on the input graph of a trained model, re-encode it, and observe the answer.

Edge ablation removes single edges. Removing the questioned edge collapses accuracy on that question, while removing an unrelated edge leaves it intact (Figure 5, left). The encoder stores edges separably.

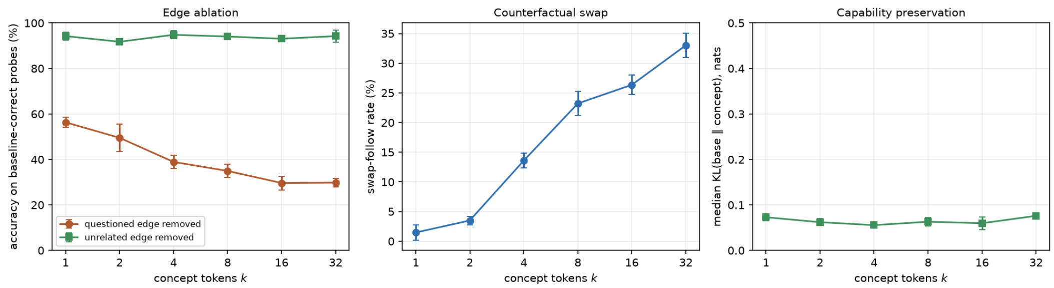


Figure 5: Causal probes across $k$ (Qwen3-0.6B, 100k corpus, 400 single-fact probes per checkpoint, 3 seeds). Left: accuracy on baseline-correct questions after removing the questioned edge or an unrelated edge. Middle: rate at which the model follows a rewired edge to the false answer. Right: median next-token KL on off-topic control tasks between the model with and without concept tokens.

Table 3: Zero-shot transfer to MetaQA-1hop (all 9,947 test questions, one seed per $k$). The Wikidata-trained checkpoints of Table 1 are evaluated without any MetaQA training. RAG verbalizes the same neighborhoods under the same token budget, without an answer guarantee.

| condition | accuracy |
|---|---|
| base (floor) | .058 |
| $k$=1 | .161 |
| $k$=8 | .235 |
| $k$=16 | .244 |
| $k$=32 | **.335** |
| RAG (reference) | .921 |

The counterfactual swap rewires the questioned edge to a type-plausible false neighbor. A model that reads the graph follows the rewiring and produces the false answer. This swap-follow rate rises from about 1% at $k$=1 to 33% at $k$=32 (Figure 5, middle). The frozen base solves only 10% of these probes from its weights, so parametric memory cannot explain the behavior. The model reads the injected graph, and it reads more of it as the token budget grows.

Injection leaves the model undisturbed elsewhere. On control tasks that mention the entity without asking about its facts, the median next-token KL between the model with and without concept tokens stays between 0.05 and 0.08 nats, flat in $k$ (Figure 5, right). Accuracy is also robust to prompt wording: across seven system prompts, PopQA accuracy of the $k$=8 model varies with a standard deviation of at most 1.1 pt.

### 5.4 Zero-shot cross-graph transfer

The featurizer consumes nothing but the frozen model's embeddings of label strings, so a trained encoder should in principle work on any labeled graph. Only an encoder that learned a general edge-to-concept mapping will actually do so. Transfer of this kind has precedent for graph retrievers, whose graph foundation models generalize to unseen datasets while the LLM still reads text [13], and graph-token systems reach new domains through multi-domain instruction tuning [14]. Zero-shot transfer of the token interface itself has, to our knowledge, not been shown before. We test this on MetaQA [21], a movie knowledge graph with 43k entities, 9 relations, and a 1-hop QA benchmark in the same style as PopQA. The graph shares nothing with Wikidata. It has no entity identifiers (the label string is the identity), its 9 relations are disjoint from Wikidata's relation vocabulary, and its questions are template-generated with multiple accepted answers. We map each relation to a natural-language label, give each incoming edge a hand-written reverse reading (`starred_actors` becomes "cast member" and "actor in"), encode each 1-hop neighborhood exactly as in §3, and score the Wikidata-trained checkpoints unchanged.

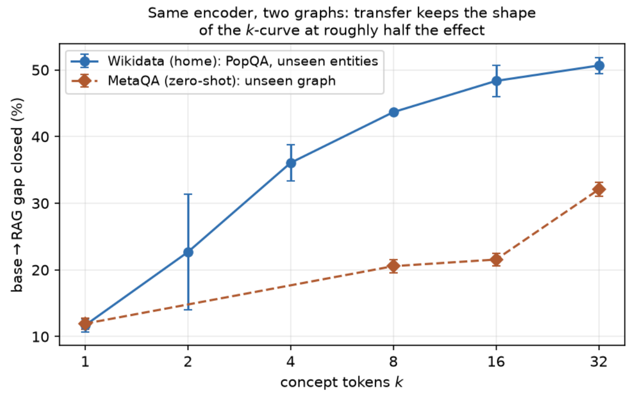


Figure 6: Share of the base-to-RAG gap closed, on the home graph (Wikidata/PopQA, 3 seeds) and zero-shot on MetaQA (one seed, Wilson CIs). The transferred curve keeps the shape of the home curve at roughly half the effect.

Table 4: Design ablations (10k corpus, $k$=8, 3 seeds each, strict held-out / PopQA). The augmentation test pools item-paired McNemar over 3 seeds; the effect is positive on average and one seed reverses on PopQA.

| ablation | variants | held-out | note |
|---|---|---|---|
| encoder capacity | 10M / 21M / **70M** / 231M | .423 / .441 / **.453** / .376 | peak at ∼70M; largest is worst |
| placement | prefix / **before** / after / replace | .453 / **.423** / .434 / .380 | **before** adopted (multi-entity) |
| prompt augmentation | off / **on** | .453 / **.472** | PopQA $+2.0$ pt; pooled $p$=$3\times10^{-36}$ |
| neighbor subsampling | **off** / on | $\Delta \approx 1$ pt | null; cached teacher is $2\times$ faster |

Injection transfers. At $k$=32, the concept tokens reach 0.335 accuracy on a graph the encoder has never seen, against a base of 0.058 (Table 3). Every $k$ beats the base under an item-paired McNemar test ($p < 10^{-10}$), and the curve stays monotone in $k$. Figure 6 compares the two graphs on a common scale, as the share of the base-to-RAG gap that the concept tokens close. Zero-shot transfer retains roughly half of the home-graph effect at every $k$ above 1, and at $k$=1 the two curves coincide. The high RAG reference (0.921) shows that the frozen model can answer in this domain when given text, so the remaining gap belongs to the transferred encoder. Together with the probes of §5.3, transfer supports the inductive claim: the encoder learned a mapping from labeled edges to concept tokens that generalizes beyond its training graph.

### 5.5 Ablations

Table 4 summarizes the design ablations. Encoder capacity peaks near 70M parameters, and the largest encoder is clearly worse at this training budget. Placement barely matters: prefix, before-entity, and after-entity cluster within noise, and only replacing the entity mention hurts. We adopt before-entity placement because it extends to prompts with several entities. Prompt augmentation gives a small gain that is significant under pooled paired testing but varies across seeds. Re-sampling teacher distractors at each step has no effect at our context budget.

## 6 Discussion

**Ceiling or steerability.** The model-size result admits two readings, and they make opposite predictions. Under a ceiling reading, a bigger base knows more of the benchmark, leaves less headroom, and more data recovers proportionally more of what remains. Under a steerability reading, a bigger frozen model needs more training signal per unit of behavioral change from $k$ soft tokens, and the data gain itself shrinks with model size. Theory supports the second reading: soft prompts can bias a frozen model's attention outputs but cannot create new attention patterns, so they elicit rather than teach [17]. Empirically, prompt-tuning gains track base-model scale in fine-tuning scaling laws [18], and the marginal utility of retrieval also depends on model scale and pretraining saturation [19]. The completed grid rejects both readings as monotone accounts. Headroom is nearly constant across the family, which rules out a pure ceiling effect, but the data gain does not decay monotonically either: it falls from $3.5\times$ at 0.6B to $2.1\times$ at 1.7B and recovers to $2.7\times$ at 4B (§5.2), and the Gemma-3 family shows the same mid-scale dip. What no configuration escapes is instability at the larger corpus: in each family, at least one 100k cell carries a seed spread above $\pm 4$ points, and which backbone is affected differs between the families. We read the two grids as evidence that the binding constraint on scaling up injection is training stability rather than headroom, with the mid-scale valley as the sharpest open question. Per-backbone recipe tuning is the experiment that would decide it.

**Limitations.** Training uses a single knowledge graph and a single language, English Wikidata. Only entity-valued relations are modeled, so literals such as dates and quantities are absent. Evaluation centers on PopQA and on held-out questions from the same generator, and the faithfulness probes cover single-fact questions. Cross-graph evaluation is zero-shot on one benchmark with one seed per $k$, and transfer with in-domain training is untested. Several 100k cells carry wide seed spreads (up to $\pm 4.4$ points, on Qwen's larger backbones and Gemma's smaller ones), so their magnitudes are less settled than the stable cells. Corpora are teacher-relative, so cross-backbone comparisons involve each model's own curriculum. Knowledge is injected for one identified entity mention per prompt, multi-entity composition is untested, and neighborhoods are 1-hop.

**Outlook.** Several extensions follow naturally. The encoder consumes embeddings, so multimodal knowledge such as entity images could enter through the same interface for natively multimodal frozen models, which is the subject of ongoing work. Wikidata carries labels in many languages, and multilingual concept tokens would test whether the learned mapping is language-independent. On the architecture side, injecting concept tokens into intermediate layers, in the spirit of deep prompt tuning [20], remains unexplored.

## 7 Conclusion

A frozen, instruction-tuned language model can be grounded in a knowledge graph through a handful of learned continuous tokens. Eight tokens more than quadruple accuracy on unseen entities at a fraction of retrieval's context cost, causal probes show that the model reads the injected graph, and the encoder transfers zero-shot to a graph it was never trained on. The injected margin shrinks as the frozen model grows, non-monotonically and in two model families, and training stability at scale emerges as the binding constraint, which makes the interaction of model scale and injection the central open question. All corpora, checkpoints, and per-item evaluation outputs are released.


## Acknowledgments and Disclosure of Funding

Drafted with substantial tooling assistance from Claude (Anthropic) for literature search, evaluation-harness engineering, and manuscript preparation. All experimental design decisions, runs, and claims were reviewed by the authors. This work received no external funding.


## References


[1] J. Barmettler, A. Bernstein, and L. Rossetto. ConceptFormer: Towards graph-native grounding of large language models via latent concept injection. In *Companion Proceedings of the ACM Web Conference 2026 (WWW Companion '26)*, pp. 587–596, 2026. Best paper award, hosting workshop. `https://doi.org/10.1145/3774905.3794653`. arXiv:2504.07624.

[2] H. Soudani, E. Kanoulas, and F. Hasibi. Fine tuning vs. retrieval augmented generation for less popular knowledge. SIGIR-AP 2024; arXiv:2403.01432.

[3] X. Wang, Z. Chen, Z. Xie, T. Xu, Y. He, and E. Chen. In-context former: Lightning-fast compressing context for large language model. EMNLP Findings 2024; arXiv:2406.13618.

[4] L. Ning, L. Liu, J. Wu, N. Wu, D. Berlowitz, S. Prakash, B. Green, S. O'Banion, and J. Xie. User-LLM: Efficient LLM contextualization with user embeddings. WWW Companion 2025; arXiv:2402.13598.

[5] L. Hebert, K. Sayana, A. Jash, A. Karatzoglou, S. Sodhi, S. Doddapaneni, Y. Cai, and D. Kuzmin. PERSOMA: Personalized soft prompt adapter architecture for personalized language prompting. arXiv:2408.00960, 2024.

[6] K. Kujanpää, P. Marttinen, H. Valpola, and A. Ilin. Efficient knowledge injection in LLMs via self-distillation. TMLR 2025; arXiv:2412.14964.

[7] S. Padmanabhan, M. O. Gul, and T. Goyal. Updating parametric knowledge with context distillation retains post-training capabilities. arXiv:2602.16093, 2026.

[8] V. Chari, G. Qin, and B. Van Durme. KV-Distill: Nearly lossless learnable context compression for LLMs. arXiv:2503.10337, 2025.

[9] C. Deng, Z. Zhang, K. Mao, S. Li, X. Huang, D. Yu, and Z. Dou. A silver bullet or a compromise for full attention? A comprehensive study of gist token-based context compression. ACL 2025; arXiv:2412.17483.

[10] W. Kim, B. Park, and W. Kim. Query-aware learnable graph pooling tokens as prompt for large language models. arXiv:2501.17549, 2025.

[11] B. Yu, W. Huang, and K. Liu. SR-KI: Scalable and real-time knowledge integration into LLMs via supervised attention. AAAI 2026, pp. 34486–34494; arXiv:2511.06446.

[12] W. Guo, X. Wang, J. Chen, L. Guo, Z. Li, and Z. Chen. ReaLM: Residual quantization bridges knowledge graph embeddings and large language models. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, pp. 3996–4007, 2026. arXiv:2510.09711.

[13] L. Luo, Z. Zhao, G. Haffari, D. Phung, C. Gong, and S. Pan. GFM-RAG: Graph foundation model for retrieval augmented generation. NeurIPS 2025; arXiv:2502.01113.

[14] H. Chen, X. Wang, J. Chao, L. Feng, and W. Zhu. A unified graph language model for multi-domain multi-task graph alignment instruction tuning. arXiv:2605.12197, 2026.

[15] G. Pantazopoulos, A. Suglia, O. Lemon, and A. Eshghi. Lost in space: Probing fine-grained spatial understanding in vision and language resamplers. NAACL 2024; arXiv:2404.13594.

[16] O. Somov, M. Chaichuk, G. Ershov, K. Vafin, M. Seleznyov, A. Panchenko, and E. Tutubalina. Breaking the chain: A causal analysis of LLM faithfulness to intermediate structures. arXiv:2603.16475, 2026.

[17] A. Petrov, P. H. S. Torr, and A. Bibi. When do prompting and prefix-tuning work? A theory of capabilities and limitations. ICLR 2024; arXiv:2310.19698.

[18] B. Zhang, Z. Liu, C. Cherry, and O. Firat. When scaling meets LLM finetuning: The effect of data, model and finetuning method. ICLR 2024; arXiv:2402.17193.

[19] K. Singh, M. Yu, V. Gangal, Z. Tao, S. Kumar, E. Liu, and S. Y. Feng. To memorize or to retrieve: Scaling laws for RAG-considerate pretraining. arXiv:2604.00715, 2026.

[20] X. Liu, K. Ji, Y. Fu, W. L. Tam, Z. Du, Z. Yang, and J. Tang. P-Tuning v2: Prompt tuning can be comparable to fine-tuning universally across scales and tasks. ACL 2022; arXiv:2110.07602.

[21] Y. Zhang, H. Dai, Z. Kozareva, A. Smola, and L. Song. Variational reasoning for question answering with knowledge graph. AAAI 2018; arXiv:1709.04071.

[22] X. Cheng, X. Wang, X. Zhang, T. Ge, S. Chen, F. Wei, H. Zhang, and D. Zhao. xRAG: Extreme context compression for retrieval-augmented generation with one token. NeurIPS 2024; arXiv:2405.13792.

[23] T. Ge, J. Hu, L. Wang, X. Wang, S.-Q. Chen, and F. Wei. In-context autoencoder for context compression in a large language model. ICLR 2024; arXiv:2307.06945.

[24] J. Mu, X. L. Li, and N. Goodman. Learning to compress prompts with gist tokens. NeurIPS 2023; arXiv:2304.08467.

[25] A. Chevalier, A. Wettig, A. Ajith, and D. Chen. Adapting language models to compress contexts. EMNLP 2023; arXiv:2305.14788.

[26] Z. Li, Y. Su, and N. Collier. 500xCompressor: Generalized prompt compression for large language models. ACL 2025; arXiv:2408.03094.

[27] M. Louis, G. Sidiropoulos, E. Kanoulas, and V. Nikoulina. PISCO: Pretty simple compression for retrieval-augmented generation. Findings of ACL 2025; arXiv:2501.16075.

[28] G. Berton et al. CompLLM: Compression for long context Q&A. arXiv:2509.19228, 2025.

[29] H. Dezéque et al. ARC-Encoder: Learning compressed text representations for large language models. arXiv:2510.20535, 2025.

[30] S. Eyuboglu et al. Cartridges: Lightweight and general-purpose long context representations via self-study. ES-FoMo Workshop, ICML 2025; arXiv:2506.06266.

[31] J. Belikova, D. Rozhevskii, D. Svirin, K. Polev, and A. Panchenko. Detecting overflow in compressed token representations for retrieval-augmented generation. EACL 2026 Student Research Workshop; arXiv:2602.12235.

[32] X. Wang, T. Isazawa, L. Mikaelyan, and J. Hensman. KBLaM: Knowledge base augmented language model. ICLR 2025; arXiv:2410.10450.

[33] C. N. dos Santos, Z. Dong, D. Cer, J. Nham, S. Shakeri, J. Ni, and Y.-H. Sung. Knowledge prompts: Injecting world knowledge into language models through soft prompts. arXiv:2210.04726, 2022.

[34] Y. Tian, H. Song, Z. Wang, H. Wang, Z. Hu, F. Wang, N. V. Chawla, and P. Xu. Graph neural prompting with large language models. AAAI 2024; arXiv:2309.15427.

[35] B. Perozzi, B. Fatemi, D. Zelle, A. Tsitsulin, M. Kazemi, R. Al-Rfou, and J. Halcrow. Let your graph do the talking: Encoding structured data for LLMs. arXiv:2402.05862, 2024.

[36] X. He, Y. Tian, Y. Sun, N. V. Chawla, T. Laurent, Y. LeCun, X. Bresson, and B. Hooi. G-Retriever: Retrieval-augmented generation for textual graph understanding and question answering. NeurIPS 2024; arXiv:2402.07630.

[37] R. Chen, T. Zhao, A. Jaiswal, N. Shah, and Z. Wang. LLaGA: Large language and graph assistant. ICML 2024; arXiv:2402.08170.

[38] Z. Zhang, Y. Yu, M. Zhang, J. Du, X. Wang, and C. Shi. Revisiting graph-tokenizing large language models: A systematic evaluation of graph token understanding. arXiv:2605.03514, 2026.

[39] D. Zhang, R. Zhou, W. Zheng, R. Fathony, B. Bruss, and C. Agarwal. When graph tokens sink: A mechanistic analysis of graph language models. arXiv:2606.03712, 2026.

[40] N. Calderon, E. Ben-David, Z. Gekhman, E. Ofek, and G. Yona. Empty shelves or lost keys? Recall is the bottleneck for parametric factuality. ICML 2026; arXiv:2602.14080.

[41] K. Sun, Y. E. Xu, H. Zha, Y. Liu, and X. L. Dong. Head-to-tail: How knowledgeable are large language models? NAACL 2024; arXiv:2308.10168.

[42] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. NeurIPS 2020.

[43] F. Petroni, T. Rocktäschel, S. Riedel, P. Lewis, A. Bakhtin, Y. Wu, and A. Miller. Language models as knowledge bases? EMNLP 2019.

[44] A. Roberts, C. Raffel, and N. Shazeer. How much knowledge can you pack into the parameters of a language model? EMNLP 2020.

[45] M. E. Peters, M. Neumann, R. L. Logan IV, R. Schwartz, V. Joshi, S. Singh, and N. A. Smith. Knowledge enhanced contextual word representations. EMNLP 2019.

[46] W. Liu, P. Zhou, Z. Zhao, Z. Wang, Q. Ju, H. Deng, and P. Wang. K-BERT: Enabling language representation with knowledge graph. AAAI 2020.

[47] R. Wang, D. Tang, N. Duan, Z. Wei, X. Huang, J. Ji, G. Cao, D. Jiang, and M. Zhou. K-Adapter: Infusing knowledge into pre-trained models with adapters. Findings of ACL 2021.

[48] A. Bordes, N. Usunier, A. García-Durán, J. Weston, and O. Yakhnenko. Translating embeddings for modeling multi-relational data. NeurIPS 2013.

[49] A. Lerer, L. Wu, J. Shen, T. Lacroix, L. Wehrstedt, A. Bose, and A. Peysakhovich. PyTorch-BigGraph: A large scale graph embedding system. MLSys 2019.

[50] X. L. Li and P. Liang. Prefix-tuning: Optimizing continuous prompts for generation. ACL 2021.

[51] B. Lester, R. Al-Rfou, and N. Constant. The power of scale for parameter-efficient prompt tuning. EMNLP 2021.

[52] G. Qin and J. Eisner. Learning how to ask: Querying LMs with mixtures of soft prompts. NAACL 2021.

[53] R. Gal, Y. Alaluf, Y. Atzmon, O. Patashnik, A. H. Bermano, G. Chechik, and D. Cohen-Or. An image is worth one word: Personalizing text-to-image generation using textual inversion. ICLR 2023.

[54] M. Tsimpoukelli, J. Menick, S. Cabi, S. M. A. Eslami, O. Vinyals, and F. Hill. Multimodal few-shot learning with frozen language models. NeurIPS 2021.

[55] A. Jaegle, F. Gimeno, A. Brock, A. Zisserman, O. Vinyals, and J. Carreira. Perceiver: General perception with iterative attention. ICML 2021.

[56] J.-B. Alayrac et al. Flamingo: A visual language model for few-shot learning. NeurIPS 2022.

[57] D. Vrandečić and M. Krötzsch. Wikidata: A free collaborative knowledgebase. Communications of the ACM, 57(10):78–85, 2014.

[58] B. Fatemi, J. Halcrow, and B. Perozzi. Talk like a graph: Encoding graphs for large language models. ICLR 2024; arXiv:2310.04560.

[59] A. Mallen, A. Asai, V. Zhong, R. Das, D. Khashabi, and H. Hajishirzi. When not to trust language models: Investigating effectiveness of parametric and non-parametric memories. ACL 2023; arXiv:2212.10511.

[60] G. Hinton, O. Vinyals, and J. Dean. Distilling the knowledge in a neural network. NeurIPS 2014 Deep Learning Workshop; arXiv:1503.02531.

[61] Qwen Team. Qwen3 model family. Technical reports, 2025; `https://huggingface.co/Qwen`.